CreditVault — Credit Card Fraud Detection

Open the website · **Download the readable learning guide**

The current app uses the **real, anonymized ULB/Worldline credit-card transaction dataset** from the original notebook. It loads a newly trained model and lets you analyze recorded transactions or a compatible CSV. PaySim and synthetic card experiments are preserved as historical work; they no longer power the main website.

Contents

CreditVault — Credit Card Fraud Detection
Start here: how data becomes a prediction
Why PCA does not invalidate this project
The four pages
The real training experiment
What SHAP can explain when features are private
What you can confidently say in a presentation
Limits to understand
Run locally
Files and reproduction
Deployment and preserved research

Start here: how data becomes a prediction

A dataset is a table of examples. Each row is a transaction. Its input columns describe the transaction numerically; its **Class** column is the known answer: 0 for legitimate and 1 for fraud.

There are two separate processes:

```
TRAINING — done before deploying the website
Earlier labeled records → fit candidate models
Middle-period records → choose the model and decision cutoff
Later-period records → measure the selected model's mistakes
Save the fitted model and its preprocessing

PREDICTION — what the website does
Load saved model → supply a complete transaction record
→ receive a fraud score → compare with cutoff → show review decision
```

The model is not a database lookup and does not receive the answer when predicting. It learns statistical relationships from training examples. Opening the app or uploading a CSV applies that learned model; it does not train it again.

Why PCA does not invalidate this project

The dataset contains 30 inputs:

Input	What it means
Time	Seconds elapsed since the source's first transaction. It is not payment duration or a verified local clock time.
Amount	The recorded transaction amount, in the dataset's supplied units.
V1–V28	Numerical components already produced by principal component analysis (PCA). Their original business meanings are private.
Class	The recorded fraud label, used for training/evaluation and excluded from prediction inputs. This is the answer, not a 31st model input.

PCA combines original numerical measurements into new numerical directions. A classifier can learn patterns in those numbers without knowing the original column names. PCA is not encryption. The public release does not provide the original features or fitted transformation, so we cannot reliably recover them or invent a mapping from merchant, location, or account balances to V1–V28.

A useful dashboard does not require someone to type 28 unfamiliar numbers. This app loads all the inputs from a saved record or a file. The normal screen shows an amount, elapsed time, model result, and recorded answer after scoring. Technical components are inside an optional expander.

The precise project description is: **a credit-card transaction fraud classifier evaluated on real anonymized transaction records, demonstrated through a web dashboard**. It is not a tool for checking an ordinary purchase using only its amount, merchant, or card number. A bank integration would need matching source features and the original preprocessing.

The four pages

1. Check a transaction

Choose a transaction from the later evaluation period, then click **Analyze transaction**. All 30 inputs are loaded automatically.

Item	What it does
Source transactions / Recorded fraud	Counts before duplicate removal, from the original source.
Selected model	The candidate chosen on validation data.
Transaction selector	Loads one unchanged real anonymized record. Example IDs are local row aliases, not bank transaction numbers.
Transaction amount / Elapsed time	The two publicly interpretable fields.
Analyze transaction	Applies the saved model without providing the recorded label.
Model fraud score	The classifier’s fraud-class output, shown on a percentage scale.

Item	What it does
Review threshold	Cutoff at or above which the model flags the transaction.
Recorded outcome	The dataset’s known answer, revealed after prediction.
Why did the model give this score?	True SHAP contributions, with private PCA components honestly labeled.
Download this transaction	Produces a CSV containing all required inputs, without the answer.

Flagged for review means the score reached the cutoff. **Not flagged** means it did not. Neither is proof of what happened; compare a recorded example with its actual label to see correct decisions and mistakes.

The 16 demonstration records include four examples of each available outcome: caught fraud, correct legitimate decisions, false alarms, and missed fraud. They are genuine model results, not fabricated examples or the complete evaluation population. Example numbers do not rank fraud risk.

2. Upload a file

Download the example CSV and upload it to practice. A valid CSV needs **Time, V1, V2, …, V28, Amount**, with finite numeric values. Column order can differ because the app restores the saved order. Component values can be negative; elapsed time and amount cannot.

Click **Analyze file** to score every row. The app shows analyzed/flagged/unflagged counts, highlights flagged rows, and offers a download with every prediction. The on-screen preview is capped at 500 rows; the download includes all analyzed rows.

Limits: 50,000 rows and 40 MB per upload. Blank files, missing columns, duplicate headers, invalid numbers, and incompatible ordinary bank statements receive explanatory errors. Uploaded labels and extra columns do not influence predictions. Replacing an upload clears previous results. The app never loads an uploaded model pickle.

Uploads run through the Streamlit server; the app does not write their records into its repository or use them for training. Use the public example records for classroom demonstrations, not personal banking statements.

3. Model results

This page measures predictions against recorded labels on **56,746 later transactions**, including **74 fraud cases**. It does not use the small demo selection to claim performance.

Read the confusion matrix as actual outcomes in rows and predicted decisions in columns:

Actual outcome	Not flagged	Flagged
Legitimate	56,666 correct legitimate decisions	6 false alarms
Fraud	19 missed fraud cases	55 caught fraud cases

Metric	Meaning	Selected model result
Precision	Of the flags, how many were actual fraud? 55 / (55 + 6).	90.1639%
Recall	Of all actual fraud, how much was caught? 55 / (55 + 19).	74.3243%
F1	Harmonic-mean balance of precision and recall.	81.4815%
Accuracy	All correct decisions divided by all test rows.	99.9559%
PR-AUC	Trapezoidal area under the precision–recall curve.	0.818762
Average precision	A separate step-wise summary of precision over recall increases.	0.818313
ROC-AUC	Ranking of fraud against legitimate transactions across cutoffs.	0.932703

The model missed 19 fraud cases. High accuracy does not mean it detected all fraud. Calling every test transaction legitimate already achieves about 99.8696% accuracy while catching zero fraud.

The precision–recall chart shows how alert correctness changes as more fraud is caught. Open the threshold expander to change the cutoff for the results page. This recalculates predictions from saved scores; it does not retrain, change the deployed prediction cutoff, or change the AUC values. Lower cutoffs can catch more fraud and create more false alarms.

Candidate	Validation F1	Test precision	Test recall	Test F1
Logistic Regression	80.0000%	94.2308%	66.2162%	77.7778%
Random Forest — selected	**84.3137%**	**90.1639%**	**74.3243%**	**81.4815%**
XGBoost	81.5534%	90.1639%	74.3243%	81.4815%

Each candidate uses its own validation-selected cutoff. Random Forest was selected before test prediction by the highest validation F1, with average precision as the tie-breaker. Its saved threshold is **0.2647505032811329**. This is a review policy on an uncalibrated score, not a proven 26.475% real-world fraud probability.

4. How it works

This page explains data, private components, training, exporting, and inference in order. It includes the actual chronological periods, model definitions, and limitations.

Term	Plain meaning
Supervised learning	Learn from examples with known answers.
Binary classification	Predict between two classes: legitimate and fraud.
Logistic Regression	Learn weighted inputs and transform their combination into a classification score.
Random Forest	Combine probabilities from many decision trees trained using varied samples/features.
XGBoost	Add trees sequentially to improve the current model.

Term	Plain meaning
Class imbalance	One class is much rarer; fraud is rare here.
Scaling	Put numerical values on comparable scales; learned only from training data for Logistic Regression.
Overfitting	Learn training-specific patterns that do not generalize to later data.
Validation	Data used to select the model and cutoff.
Test evaluation	Measure the frozen choices on later records.
Calibration	Check whether reported scores match observed outcome frequencies. It has not been performed here.
SHAP	Explain how inputs contribute to the trained model’s output relative to a reference.

The real training experiment

Source: ULB / Worldline Credit Card Fraud Detection, the public benchmark of European cardholders’ transactions from two days in September 2013.

The verified source has **284,807 rows and 492 fraud labels**. We remove **1,081 exact duplicate records**, leaving **283,726 transactions and 473 fraud labels**. Fraud duplicates are removed too; the changed count is documented rather than hidden.

Sort by elapsed time and split around 60% / 20% / 20%, keeping equal timestamps together:

Period	Elapsed seconds	Rows	Fraud	Use
Train	0–120,392	170,235	342	Fit model parameters and any scaling.
Validation	120,393–145,233	56,745	57	Choose candidate and cutoff.

Period	Elapsed seconds	Rows	Fraud	Use
Later test	145,234–172,792	56,746	74	Report the final measured performance.

Every deduplicated training transaction is retained. This experiment does not undersample normal transactions, fabricate fraud, use SMOTE, or fit a new PCA transformation. Validation and test keep their natural class imbalance. Labels are excluded from all model inputs.

The fixed candidates are:

- Logistic Regression: training-only StandardScaler, balanced class weights, up to 2,000 iterations.
- Random Forest: 200 trees, maximum depth 16, at least 2 samples per leaf, square-root feature sampling, balanced weights within each tree's sample.
- XGBoost: 400 trees, depth 4, learning rate 0.05, minimum child weight 3, row sampling 0.85, feature sampling 0.9, regularization 5, and square-root negative/positive training class ratio as the fraud weight.

All use seed 42 where applicable; tree training uses two workers. These are specified experimental settings, not an exhaustive hyperparameter search. No candidate emitted a fitting warning. Model selection is frozen before later-period scores are inspected.

The old notebook trained on 787 rows after heavy undersampling and evaluated a balanced 197-row holdout. Its score is not directly comparable with these results on a naturally imbalanced 56,746-row later test. Its exact original workflow remains preserved under `artifacts/`.

What SHAP can explain when features are private

The current Random Forest explanation uses 64 genuine earlier training rows as its background. Its reference value plus all 30 contributions reconstructs the model's probability output, checked

numerically. The app charts the eight largest contributions and describes the three largest.

A positive contribution raises the model score; a negative one lowers it. For example, “V14 increased the score” is a valid statement about model behavior. “V14 means an unusual merchant” would be invented; the public source does not reveal that meaning.

SHAP describes the calculation, not why a crime occurred. Related anonymous components and the provider’s undisclosed original preprocessing limit business interpretation. If a different saved candidate is used, the explanation reports its actual units; Logistic Regression uses log-odds, while the tree explanations here use probability output.

What you can confidently say in a presentation

> “My system detects potentially fraudulent credit-card transactions using a real anonymized benchmark. I train and compare three classifiers, select a model on validation data, and measure its performance on later transactions. The website loads complete records automatically, gives a score and review decision, and shows measured errors. PCA protects original feature meanings in the public release; the classifier can still use those numerical patterns.”

A short demonstration order is: **How it works → Check a transaction → Model results → Upload a file**.

Questions worth preparing:

- **Why are the features anonymous?** Original details are withheld for confidentiality; the public release contains transformed components.
- **How can the model learn without readable names?** It learns numerical relationships between inputs and labeled examples, not the English meanings of column names.
- **Why can’t I enter only a merchant and amount?** That does not supply the 28 private components the trained model requires. We do not have their original transform.

- **Is every flagged transaction fraud?** No. The test includes 6 false alarms and 19 missed fraud cases.
- **Does uploading data train the model?** No. It applies the saved model. Training is a separate reproducible script.
- **Is a score of 90% a guaranteed fraud probability?** No. Scores are uncalibrated and need population-specific validation.

Limits to understand

The dataset is historical, covers only two days, and has just 74 fraud cases in this test period. One additional caught or missed case moves recall by about 1.35 percentage points. This is not evidence of current-bank or universal performance.

The original PCA fitting procedure and underlying meanings cannot be audited from the public release. No customer identifiers are provided, so we cannot establish evaluation on entirely unseen customers. The same public source was already used by the earlier notebook; this is a new chronological experiment, not an untouched external-bank audit. It does not connect to bank accounts, authorize payments, or prove someone's intent.

Run locally

Use Python 3.12 and the pinned model dependencies:

```
git clone https://github.com/me-vishnurnair/Machine-learning-project.git
cd Machine-learning-project
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
streamlit run app.py
```

Windows PowerShell activation: `.venv\Scripts\Activate.ps1`.

```
python -m pip install -r requirements-dev.txt
python -m pytest -q
```

No training, API keys, account credentials, or full-source download is needed to run the app. Model loading uses `st.cache_resource`; data/score/explanation loading uses `st.cache_data`. Native lazy tabs keep unused pages out of the initial render.

Files and reproduction

File	Purpose
app.py	Public entry point; imports the focused credit-card dashboard.
credit_dashboard.py	Four pages, readable results, uploads, and explanations.
credit_core.py	Exact 30-feature contract, input validation, CSV scoring, model loading, SHAP.
train_credit_model.py	Verified source, deduplication, chronological splitting, real model fitting and export.
credit_artifacts/model.pkl	Selected fitted pipeline.
credit_artifacts/metadata.json	Dataset, recipe, models, thresholds, metrics, versions, limitations.
credit_artifacts/feature_columns.json	Exact input order.
credit_artifacts/requirements-model.txt	Scientific package versions for compatible model loading.
credit_artifacts/demo_transactions.csv	16 actual later records with recorded outcomes/scores for replay.
credit_artifacts/example_upload.csv	Ready-to-use inputs with no label or saved prediction.
credit_artifacts/evaluation_scores.npz	Full later test labels and all three candidate scores.
credit_artifacts/test_transactions.parquet	Full later records for independent evaluation replay.
credit_artifacts/background.csv	Actual training references for SHAP.
tests/test_credit_*.py	New inference, evaluation, explanation and UI checks.

The full source is kept outside the repository. Its SHA-256 is:

```
76274b691b16a6c49d3f159c883398e03ccd6d1ee12d9d8ee38f4b4b98551a89
```

Source download and fingerprint evidence are recorded in the original provenance document. Dataset licensing is not inferred from the mirror’s code license. To reproduce into a new empty directory:

```
python train_credit_model.py --data /path/outside/repository/creditcard.csv --
        output /path/new-credit-export
```

The trainer rejects mismatching source bytes, fits every candidate, chooses on validation, evaluates later rows, saves the selected pipeline, and verifies reloaded demo scores. `CREDIT_ARTIFACT_DIR` can point to a complete matching trusted export. Never load an untrusted pickle.

Deployment and preserved research

The public Streamlit app uses this repository's `main` branch and `app.py`. The complete model bundle is committed; pushing updates triggers Streamlit Community Cloud. For a new deployment, connect the repository, select `main` and `app.py`, and choose Python 3.12. No secrets are required.

The previous experiments and notebooks are preserved. Run `streamlit run presentation_dashboard.py` for the earlier PaySim dashboard, `streamlit run card_dashboard.py` for the separate Sparkov study, or `streamlit run legacy_dashboard.py` for the original research interface. Their data and scores must not be presented as the current ULB model's results.

Archived learning materials: PaySim text guide · PaySim PDF archive.